

PowerScale: Energy-Efficient Geo-Distributed Model Training with Federated Datacenter Power

Talha Mehboob

University of Massachusetts Amherst
Amherst, Massachusetts, USA
tmehboob@umass.edu

Michael Zink

University of Massachusetts Amherst
Amherst, Massachusetts, USA
mzink@cas.umass.edu

Zhe Xu

University of Massachusetts Amherst
Amherst, Massachusetts, USA
zhxu@umass.edu

David Irwin

University of Massachusetts Amherst
Amherst, Massachusetts, USA
irwin@ecs.umass.edu

Abstract

The power demands of large-scale AI training increasingly exceed the capacity of any single data center, making geo-distributed training across power-constrained sites a practical necessity. Prior work optimizes such training mainly for time-to-accuracy, relying on single-tier aggregation, where every site exchanges model updates directly with a central aggregator over the wide-area network (WAN) each synchronization round, without accounting for the energy required to reach convergence. Single-tier aggregation, however, is fundamentally energy-inefficient for three reasons. First, each round is bottlenecked by the most distant participant, forcing faster sites to draw near-idle GPU power while they wait. Second, every site transmits its full model update over long-haul WAN links, such that beyond a certain site count, communication energy dominates the compute budget. Third, a fixed synchronization frequency pays the same communication cost throughout training, even in later stages where parameter updates shrink and further synchronization yields diminishing benefits.

To address these inefficiencies, we present **POWERSCALE**, a hierarchical aggregation system that exploits the natural latency hierarchy of wide-area networks. **POWERSCALE** organizes sites into regional clusters and applies a Sync-Async synchronization modality: sites synchronize frequently with a nearby cluster aggregator over fast local links (synchronous cluster tier), while cluster aggregators push pre-aggregated updates asynchronously to a global aggregator over the WAN (asynchronous global tier). **POWERSCALE** forms clusters based on both network proximity and power availability, and implements an adaptive synchronization policy which further reduces communication energy by adapting how often clusters synchronize to training progress. This structure shortens the synchronization barrier that idles GPUs and replaces per-site WAN transmissions each round with fewer, pre-aggregated transmissions at a lower frequency, reducing long-haul traffic. We evaluate **POWERSCALE** at 100-site scale in a Flower-based simulation environment. **POWERSCALE** matches or slightly improves the time-to-accuracy of single-tier baselines while reducing energy consumption by up to 3.9 $\times$.

1 Introduction

The computational and power demands of large-scale AI training increasingly exceed what any single data center can supply [5, 29, 35]. U.S. data centers consumed 176 TWh of electricity in 2023, up

from 76 TWh in 2018, an increase of 100 TWh in five years which is projected to grow by an additional 149–404 TWh by 2028 [40]. Over the same 2018–2023 window, total U.S. electricity generation grew by only 2.3 TWh [43]. **Figure 1** shows the resulting divergence: AI-driven energy demand is growing by an order of magnitude faster than the grid that must supply it.

The shortfall extends beyond raw generation capacity; it is fundamentally a deliverability bottleneck. Even when new capacity is provided, grid interconnection constraints prevent immediate deployment. Currently, over 2,300 GW of generation and storage remains stalled in transmission interconnection queues [34]. Furthermore, the median project now waits roughly five years from request to operation, up from under two years a decade ago [32]. Consequently, new power cannot be provisioned on the timelines or at the locations required by AI training workloads, leaving individual datacenters strictly power-constrained.

When faced with strict power constraints at individual sites, operators often throttle workloads via power capping [36] or demand response [32]. However, this approach idles provisioned GPUs and prolongs training times. Aggregating the available capacity across *multiple* power-constrained sites mitigates this underutilization. Industry architectures are increasingly adopting this paradigm: Microsoft’s “AI superfactory” interconnects datacenters across different U.S. states via dedicated wide-area networks (WANs) to train single models as a unified virtual cluster [15], while Google distributes the training of its largest foundation models across multiple campuses and metropolitan areas [13]. Consequently, geo-distributed training across power-constrained sites is emerging as standard practice for large-scale AI workloads [36].

However, geo-distribution introduces substantial communication overhead. Depending on site placement, synchronization rounds may cross regional or long-haul WAN links whose latency and bandwidth are much worse than within a single data center. This communication penalty can severely degrade, or entirely negate, the computational benefit provided by aggregate power. Consequently, realizing a net performance benefit from geo-distributed training is non-trivial. Prior work has demonstrated its viability: by strategically selecting the number and location of participating sites, it navigates the fundamental trade-off between distributed power capacity and communication delay [29].

Prior work, however, evaluates geo-distributed training almost entirely through the lens of *performance*: it optimizes time-to-accuracy and treats site power as a capacity constraint, without directly accounting for the total *energy* consumed to reach convergence. Our central insight is that this performance benefit is obtained at a large and largely avoidable energy cost because prior work largely relies on single-tier aggregation, where every site is exchanging model updates directly with a central aggregator over the wide-area network (WAN) each synchronization round. Single-tier, all-to-one aggregation is fundamentally energy-inefficient due to three compounding factors. First, synchronization barriers force faster sites into an idle state in which their GPUs continue to draw roughly 25–35% of peak power [18] while waiting for the most distant participant, so each round’s energy overhead scales with the latency of the slowest site in the pool of training sites. Second, every site transmits full gradient updates directly over WAN links; as the site count grows, this redundant long-haul traffic dominates the energy budget, and beyond a critical threshold adding more participants actively *degrades* energy efficiency because the communication penalty outweighs the computational gain. Third, a static synchronization frequency incurs a constant communication penalty even during later stages of training where updates yield diminishing convergence benefits [1].

Building on this insight, POWERSCALE exploits a structural property of geo-distributed networks: intra-regional links exhibit round-trip latencies one to two orders of magnitude lower than inter-continental links. POWERSCALE is a hierarchical aggregation system that organizes sites into regional clusters mirroring this physical network structure and optimizes energy through three *energy-efficiency policies*. First, a Sync-Async synchronization modality applies synchronous aggregation within each cluster to keep cluster models coherent at low cost, while asynchronous aggregation across clusters reduces the WAN-scale waiting barrier that would otherwise idle entire clusters. Second, power-aware agglomerative clustering groups sites on a joint criterion fusing network proximity with available power, producing clusters whose member sites have both low latency and high compute throughput. Third, an adaptive synchronization frequency assigns each cluster a global synchronization rate proportional to its communication cost and adaptively tapers WAN exchanges as training matures and parameter update magnitudes decrease. Together, these policies confine the majority of communication to fast regional links and reduce costly global exchange as the model converges, substantially lowering both idle and transmission energy while modestly improving time-to-accuracy.

Realizing these policies introduces significant challenges: POWERSCALE must cluster sites according to their power capacities and network proximity, determine the appropriate number and composition of clusters, modulate global synchronization frequency per-cluster and over time, and balance synchronous against asynchronous inter-cluster aggregation without introducing model drift. Prior hierarchical federated learning explores similar structures but targets mobile edge computing, optimizing for privacy and battery life under relatively static conditions [9]; it does not address the heterogeneous power capacities, WAN-bound communication, and energy objectives of large-scale data center training.

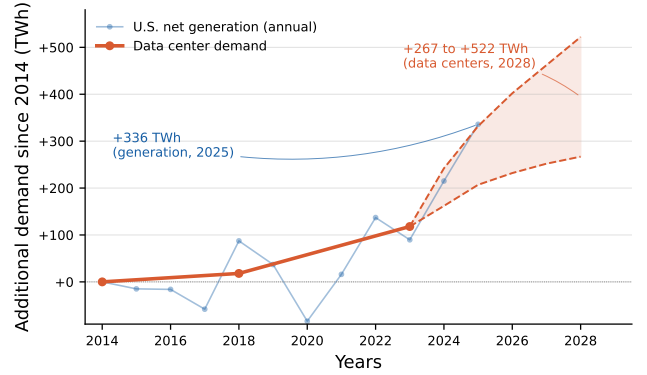


Figure 1: Growth in U.S. data center electricity demand against the growth in total grid generation [40, 43].

We hypothesize that a hierarchical aggregation structure mirroring the physical network, grouping geo-distributed sites into regional clusters with appropriate synchronization policies, can substantially reduce the energy cost of distributed training relative to a single-tier, all-to-one aggregation, while maintaining comparable time-to-accuracy. In evaluating this hypothesis, we make the following contributions:

Quantify energy inefficiency of single-tier aggregation. We show that single-tier geo-distributed training optimized for time-to-accuracy is highly energy-inefficient in three ways: idle waiting at synchronization barriers, excessive WAN transmissions, and convergence-agnostic synchronization frequency.

Energy-efficient policies for geo-distributed training. We design POWERSCALE, which introduces three energy-efficiency policies: (1) a Sync-Async communication modality that eliminates WAN-scale waiting barriers while preserving intra-cluster model coherence (2) agglomerative clustering on a joint network-proximity and power-availability criterion and (3) an adaptive synchronization frequency that assigns per-cluster rates and tapers global WAN exchanges over training.

Implementation and evaluation. We evaluate POWERSCALE using a Flower-based [6] simulation at 100-site scale. Our policies reduce the energy-to-accuracy by 3.9 $\times$ relative to single-tier baselines while maintaining or slightly improving time-to-accuracy across multiple ML workloads.

2 Background

In this section, we first discuss the power constraints that make geo-distributed training necessary (Section 2.1), then introduce geo-distributed model training and the single-tier aggregation used by prior work (Section 2.2). We then define the energy-inefficiency problem with the single-tier aggregation (Section 2.3).

2.1 Power Constraints at a Single Site

A data center’s computational throughput depends on several factors, including its hardware resources, cooling capacity, and network bandwidth. Increasingly, the fundamental constraint is power,

because the electricity a site can draw is capped by its grid connection. This constraint limits how many accelerators a site can run, regardless of how much hardware is physically installed [29, 32].

To use data center capacity efficiently, operators often oversubscribe their power budgets, provisioning more hardware than the site could run at full power. As a result, a training job must share the available power with co-located workloads, which compete for the same grid allocation [36]. The power available for training at site k is therefore the time-varying residual $P_{\text{cap},k} - P_{\text{utilized},k}(t)$, where $P_{\text{cap},k}$ is the site's rated power capacity and $P_{\text{utilized},k}(t)$ is the power consumed by other workloads at time t . The residual power fluctuates

and differs substantially across sites [36].

When the residual power of a site drops, it can sustain fewer active accelerators, reducing its training throughput proportionally. To avoid exceeding the available power, the workload is throttled, either by capping power consumption or shedding load through demand response, which idles provisioned hardware and delays training. This power shortage is not isolated to a single site. With power demand increasingly outpacing supply, no single site can reliably provide the sustained power that large-scale model training demands. Moreover, new grid capacity cannot be brought online quickly enough to close the gap, as interconnection queues now impose multi-year delays [34]. Pooling many such sites, however, aggregates far more power and compute capacity than any single site can provide. This motivates the shift to geo-distributed training.

2.2 Geo-Distributed Model Training

Geo-distributed model training refers to the practice of training a single machine learning model across multiple data centers that are geographically dispersed, potentially spanning different cities, states, or continents. Traditional distributed training assumes co-located nodes connected by high-bandwidth, low-latency interconnects, such as NVLink within a server and InfiniBand across servers within a cluster [4]. Geo-distributed training instead operates over wide-area networks. These networks have high propagation delays (tens to hundreds of milliseconds), limited and variable bandwidth, and heterogeneous compute across sites.

Existing systems geo-distribute training using two primary paradigms. Industry deployments, such as Microsoft's AI superfactory [15], partition the *model* across sites. This approach extends pipeline or tensor parallelism over dedicated inter-site links to pool massive hardware arrays. However, model parallelism requires constant, high-bandwidth communication between sites. It also typically necessitates transporting massive volumes of training data across the network to the compute sites. These requirements make pipeline parallelism highly inefficient and brittle over a standard wide-area network (WAN).

In contrast, Federated Learning (FL) partitions the *data* across sites [27]. FL keeps data locally situated primarily to preserve user privacy. Our setting adopts this data-parallel architecture, but for an entirely different reason. In power-constrained environments, data parallelism is structurally better than model parallelism. Because each site computes independently on its local data partition and only synchronizes model parameters periodically, this decoupled approach tolerates high WAN latencies and power fluctuations.

Following this paradigm, we assume the training dataset is already distributed and locally available at the participating sites. While this mirrors the mechanical structure of FL, we do not partition data for privacy. Instead, sites participate because they possess spare power capacity (Section 2.1).

Under data parallelism, the synchronization mechanism, namely how often, with whom, and over what network path model updates are exchanged, is the main challenge, as the WAN introduces communication costs that are orders of magnitude higher than those in co-located settings [22]. Existing systems organize this synchronization as *single-tier aggregation*, where every participating site exchanges its model update directly with one central aggregator over the WAN each round [4]. A round proceeds synchronously. The aggregator broadcasts the current global model, each site trains locally for a fixed number of steps, and the aggregator averages the returned updates into a new global model. The time to transfer a model update of size D_m between a site k and the aggregator is

$$T_{k,\text{comm}} = \frac{2d_k}{c} + \frac{D_m}{B_k}, \quad (1)$$

where d_k is the physical distance between them, c is the propagation speed of the signal, and B_k is the available bandwidth on the link [29]. The first term is the round-trip propagation delay; the second is the transmission time. A round cannot complete until every site has returned its update, so the round's communication time is set by the slowest participant, $T_{\text{comm}} = \max_{k \in S} T_{k,\text{comm}}$.

Prior systems address this communication cost by tuning which and how many sites participate [29]. Others reduce the data exchanged per round, through gradient compression [23] or additional local steps between synchronizations [12]. In all cases, however, the structure remains single-tier: every site synchronizes directly with a single global aggregator.

2.3 Inefficiency of Single-Tier Aggregation

Single-tier aggregation techniques are designed to maximize throughput and minimize time-to-convergence, and are commonly deployed in edge or mobile environments where transferred model updates are small (kilobytes to megabytes) [27] and wide-area bandwidth limitations remain manageable. In geo-distributed datacenter training, however, model updates are massive, often gigabytes in size [12]. At this scale, WAN communication becomes a major bottleneck, degrading both performance and energy efficiency. Prior work primarily optimizes for training time and overlooks the energy overhead of long-haul communication and idle resources.

To formalize this overhead, we consider the energy cost of a single training round. Each site passes through three activity states: computing, communicating, and idling at the synchronization barrier while waiting for all other participants to report. The accelerators remain powered in each state but draw power at different rates. Letting $t_{\text{comp},k}$, $t_{\text{comm},k}$, and $t_{\text{idle},k}$ denote the time site k spends in each state, its round energy is

$$E_k = P_{\text{frac},k} (t_{\text{comp},k} + \alpha t_{\text{comm},k} + \beta t_{\text{idle},k}), \quad (2)$$

where $P_{\text{frac},k}$ is the normalized power allocation at site k , proportional to the power it draws while computing (Section 2.1), and $\alpha, \beta \in (0, 1)$ scale this draw during communication and idling,

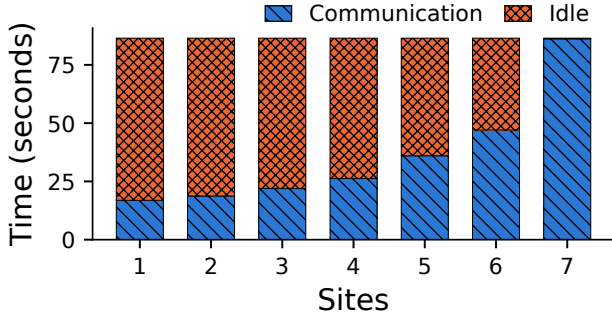


Figure 2: Per-site communication and idle time in one synchronous round, ordered by increasing distance from the aggregator.

since accelerators consume less power when not actively computing. Summed over all sites and aggregators, the total decomposes into $E = E_{\text{compute}} + E_{\text{comm}} + E_{\text{idle}}$.

This decomposition reveals two properties that shape our design. First, compute energy is essentially fixed: it reflects the work needed to reach a target accuracy, which reorganizing sites does not change. Power throttling (Section 2.1) stretches $t_{\text{comp},k}$ when $P_{\text{frac},k}$ decreases, but their product remains constant. Second, because every term is power multiplied by time, reducing the duration of communication or idling reduces the corresponding energy directly. The reducible overhead is therefore the communication and idle energy, and single-tier aggregation inflates exactly these two components in three ways.

First, it inflates idle energy (the $\beta P_{\text{frac}} t_{\text{idle}}$ term of Eq. 2). Each synchronous round cannot finish until the most distant site reports, so faster sites sit at the barrier with their accelerators powered but idle. This energy overhead grows with the latency spread across sites, which is large over the WAN. Figure 2 illustrates this directly. Sites near the aggregator finish their transfers well before the most distant site, yet none can proceed until the slowest participant arrives. In a single-tier design, this waiting repeats every round, paying for accelerator power with no training progress.

Second, it inflates communication energy ($\alpha P_{\text{frac}} t_{\text{comm}}$). Every site sends its full update over the WAN each round. Because nearby sites transmit independently rather than combining their updates regionally first, many of these transmissions are redundant: the same information could reach the aggregator in a single pre-aggregated message per region. This redundant traffic grows with site count, and beyond a critical number, adding sites degrades energy efficiency even as it continues to improve time-to-accuracy.

Third, communication energy is spent even when it yields little benefit. A fixed synchronization frequency has the same per-round WAN cost throughout training, including late stages where gradients are small and synchronization barely improves accuracy.

Existing single-tier techniques do not address these overheads. Backup workers [8] mitigate stragglers but require spare compute capacity that power-constrained sites lack. Collective operations like ring all-reduce [39] assume uniform, high-bandwidth interconnects rather than slow, heterogeneous WAN links. Asynchronous

methods remove the synchronization barrier but still incur a full WAN round-trip per step and introduce model drift. Recent work optimizes site selection [29] or synchronization frequency [12], reducing some overhead, but every site still reports directly to a global aggregator, leaving these energy inefficiencies unresolved.

3 POWERSCALE Design

To address the energy inefficiencies of single-tier geo-distributed training in power-constrained data centers, we present POWERSCALE. POWERSCALE utilizes hierarchical aggregation, exploiting the spatial heterogeneity of wide-area networks: links between data center sites in the same region offer bandwidth and latency an order of magnitude better than inter-continental ones [22]. By confining most synchronization to these fast regional links, hierarchical aggregation reduces communication energy, because each cluster forwards a single pre-aggregated update over the WAN in place of many individual ones. It also reduces idle energy, because sites wait only for others in their own cluster rather than for all participants.

Hierarchical aggregation is not a new concept. Hierarchical Federated Learning (HFL) [9, 24, 25] applies this principle in mobile and edge computing, where it relieves communication bottlenecks at the central server and accommodates resource-constrained edge devices. POWERSCALE borrows the same structural intuition but applies it in a different context and with a different objective: minimizing energy inefficiencies under strict power constraints. Unlike HFL, where edge devices can sleep when idle and consume negligible power, datacenter accelerators draw substantial power even when stalled at a synchronization barrier, making every wait an energy cost rather than a delay. Moreover, each site in our setting contributes power, and reaching more distant sites to pool additional power incurs proportionally higher communication energy, a trade-off between power and communication cost that does not arise in the edge setting.

Implementing this hierarchical structure for energy-efficient geo-distributed training raises three technical challenges, each corresponding to one of the energy overheads identified in Section 2.3. First, the hierarchy introduces synchronization at two levels, within clusters and between clusters, and how each level synchronizes directly determines idle energy. If all clusters must wait for the slowest one before the global model is updated, the energy overhead due to straggler idling of single-tier aggregation reappears at the cluster level (Section 3.1). Second, how sites are grouped into clusters determines both communication and idle energy simultaneously. A poor grouping can place nearby sites in different clusters, reintroducing redundant long-haul traffic, or strand a low-power site in a fast cluster, creating a straggler that idles every other member (Section 3.2). Third, how often clusters synchronize presents a trade-off: more frequent synchronization incurs higher WAN communication energy, while less frequent synchronization leads to model drift and slower convergence (Section 3.3).

POWERSCALE addresses these challenges through an energy-efficiency policy that resolves each one dynamically at runtime, adapting to each site’s available power and the model’s convergence progress rather than a fixed configuration set before training begins.

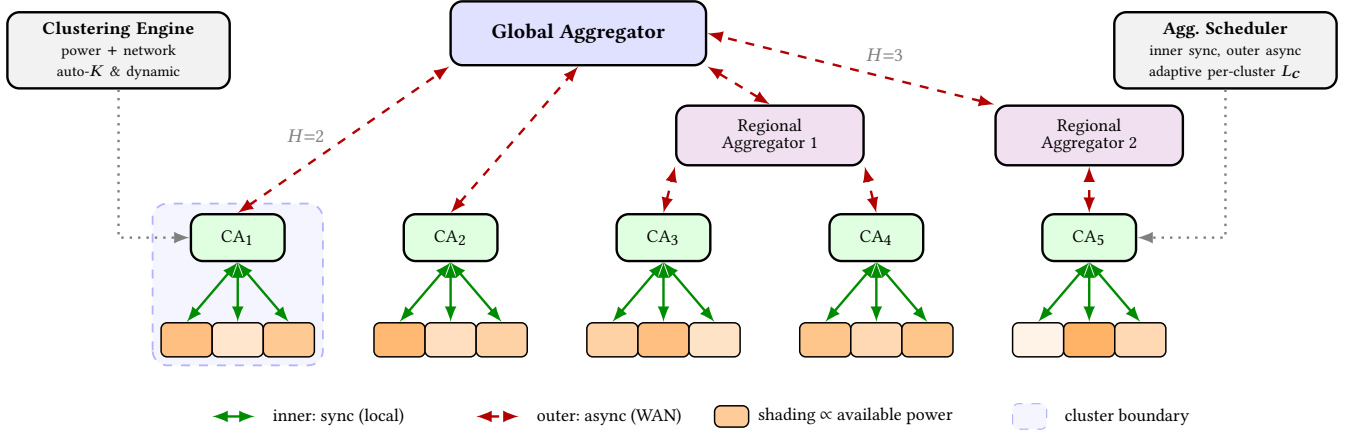


Figure 3: POWERSCALE’s architecture. The left side depicts the default two-tier ($H=2$) topology; the right illustrates a three-tier ($H=3$) extension using regional aggregators.

Figure 3 shows the resulting architecture. We discuss each component in detail in the following subsections, beginning with the synchronization modality that forms the foundation of the system.

3.1 Synchronization Modality

Given N sites, POWERSCALE organizes them into K regional clusters, each managed by a *cluster aggregator* (CA), with the CAs reporting to a single *global aggregator* (GA). Model training uses a two-tier update strategy: frequent updates occur at the *cluster level* (within each cluster), while infrequent updates occur at the *global level* (across all clusters).

In Figure 3, these are the green intra-cluster links and the red inter-cluster links, respectively.

The first design decision is how each tier synchronizes. Each tier can operate either synchronously, where all participants must report before aggregation proceeds, or asynchronously, where updates are merged as they arrive. This yields four configurations.

- **Sync-Sync.** Both tiers are synchronous. Models stay fully coherent, but faster sites and faster clusters must wait for the slowest one at both levels.
- **Sync-Async.** The cluster tier is synchronous; the global tier is asynchronous. Clusters stay coherent internally, while they no longer wait for each other at the global tier.
- **Async-Sync.** The cluster tier is asynchronous; the global tier is synchronous. Sites within a cluster can finish and be aggregated at different times, but clusters still wait for others to send updates.
- **Async-Async.** Both tiers are asynchronous. No participant waits for any other, but updates can be aggregated using stale information at both levels, which slows convergence.

At each tier, the synchronization mode is chosen based on how much participants differ in their completion times.

Cluster tier (intra-cluster). In each round, the CA broadcasts its current model, sites train locally on their data partitions, and the CA averages the returned updates. POWERSCALE aggregates the model updates at the cluster tier *synchronously*. This is because the sites within a cluster are geographically proximate and share similar

network characteristics, so their completion times are close to each other. The wait imposed on faster sites is therefore short, and the idle energy accumulated during that wait is small. Synchronous aggregation also keeps the cluster model coherent. Running the cluster tier asynchronously can reduce the wait further, but since completion times are similar, the marginal energy savings would come at the cost of model coherence.

Global tier (inter-cluster). At the global tier, the GA aggregates the model updates from clusters *asynchronously*. Clusters connect to the global aggregator over the WAN, and their completion times differ significantly due to differences in link latency and aggregate compute. Waiting for the slowest cluster each round would reintroduce idle energy overhead at the cluster level, negating the benefit of the hierarchy. Instead, each CA pushes its update to the GA as soon as it finishes, without waiting for other clusters. The GA merges arriving updates into the global model, weighting each by the contributing cluster’s size. To handle updates arriving at different times, the GA maintains a buffer of size B . Each time B cluster updates have accumulated, the GA aggregates them into the global model, weighting each update inversely by its staleness, the number of global rounds elapsed since it was computed. This ensures that more recent updates have greater influence on the global model, while older updates are not discarded but down-weighted. Because the hierarchy replaces N individual WAN transmissions per round with only K cluster-level transmissions every few rounds, the total long-haul communication drops substantially, directly reducing communication energy.

POWERSCALE therefore applies the **Sync-Async** configuration: synchronous at the cluster tier and asynchronous at the global tier. This confines waiting to the cluster tier, where the cost is low, and removes it from the global tier, where the cost is high. We evaluate all four configurations in Section 5.3. We integrate this Sync-Async modality as the core communication loop of our runtime policy (Algorithm 3.1, Lines 10–18).

3.2 Clustering Policy

With the synchronization modality established, the second component of POWERSCALE’s policy determines how sites are grouped into clusters. The grouping determines both communication and idle energy: placing nearby sites in the same cluster confines most traffic to fast regional links, while balancing power across clusters prevents any one cluster from being slowed by a low-power site. POWERSCALE’s clustering policy addresses three decisions: the clustering algorithm, the criterion on which sites are grouped, and how the grouping is maintained as conditions change during training. **Clustering Algorithm.** POWERSCALE supports two clustering algorithms, selected based on the deployment scenario.

K-means [26] partitions sites along the network-distance axis, minimizing within-cluster variance in distance to the global aggregator. It is preferred when the cluster count is known in advance, for example when it corresponds to fixed geographic regions, and when low computational overhead during re-clustering is a priority.

Agglomerative clustering [30] is POWERSCALE’s default for deployments where the cluster count is not fixed or where site conditions change frequently. It operates on a combined feature that fuses network distance and power availability,

$$f_k = w_d \tilde{d}_k + w_p (1 - \tilde{P}_{\text{frac},k}), \quad (3)$$

where $\tilde{d}_k$ and $\tilde{P}_{\text{frac},k}$ are the normalized distance and power fraction of site k , and w_d and w_p are weights that control the relative importance of the two features (we use $w_d=0.7$, $w_p=0.3$). Starting from singleton clusters, the algorithm repeatedly merges the closest pair of clusters under f_k until the target count is reached. The merge sequence produces a dendrogram, a hierarchical tree of clusters that enables efficient re-clustering at runtime: cutting the tree at a different height yields a new grouping without re-solving from scratch. The nested structure naturally supports multi-level hierarchies.

Clustering Criterion. POWERSCALE groups sites jointly on two properties: network distance and available power. Network distance d_k determines the communication latency between a site and its cluster aggregator, as given by the delay model of Eq. 1. Available power fraction $P_{\text{frac},k} \in (0, 1]$ determines a site’s available compute throughput (Section 2.1). POWERSCALE combines these into a score that reflects the compute a site contributes relative to the communication cost it introduces,

$$\text{Score}(k) = \frac{P_{\text{frac},k}}{T_{k,\text{comm}}}, \quad (4)$$

where $T_{k,\text{comm}}$ is the transfer time from site k to the global aggregator for a reference update size (Eq. 1). This score is used to rank sites when determining active participation (Equation 3.2).

The two properties serve complementary roles. Grouping sites by proximity keeps intra-cluster transfer times short, directly reducing communication energy. Balancing power across clusters ensures that no cluster is consistently paced by a low-power site, reducing idle energy at the synchronization point. Clustering on network distance alone can strand a low-power site inside an otherwise fast cluster, raising idle energy. Clustering on power alone can group distant sites together, inflating communication cost.

Cluster Count and Size. Deciding the number of clusters K involves a trade-off. A small K produces large clusters that amortize global synchronization across many sites, but large clusters increase

the probability that at least one site finishes significantly later than the others, extending the synchronization wait. A large K produces smaller, more homogeneous clusters, but increases the number of cluster aggregators that must transmit over the WAN each global round, raising communication energy.

POWERSCALE selects K automatically by sweeping candidate values up to $\min(2\sqrt{N}, N/2)$ and estimating per-round cost for each under the delay model of Eq. 1. K with the lowest estimated cost is selected, favoring larger values on near-ties for greater parallelism.

Individual cluster sizes vary because POWERSCALE groups sites by feature similarity (Eq. 3) rather than enforcing equal counts. To prevent any cluster from growing to a size that slow completions become likely, POWERSCALE bounds each cluster’s size around the mean. Assuming independent completion events across sites, the probability that at least one site in a cluster of size N_c causes a delay is

$$P(\text{delay}) = 1 - (1 - p_s)^{N_c}, \quad (5)$$

where p_s is the per-site probability of a slow completion in a given round. This probability grows with N_c , so POWERSCALE limits cluster sizes to keep it below an acceptable threshold.

Dynamic vs. Static Clustering. A *static clustering* derived from conditions at the start of training is optimal only for those initial conditions. Available power varies over time as co-located production workloads compete for grid capacity (Section 2.1). As power budgets shift, a grouping that was well-balanced initially may place power-reduced sites into clusters where they extend the synchronization wait, increasing idle energy, or may exclude high-powered sites that have become available, leaving compute capacity unused.

POWERSCALE therefore presents a *dynamic clustering* technique which re-evaluates the clustering during training, triggered periodically and whenever a significant change in site power is detected. Using the existing dendrogram, POWERSCALE updates the current grouping incrementally, splitting clusters that have grown too large and merging those that have become too small, re-solving from scratch only when incremental updates are insufficient. This approach keeps the grouping aligned with current conditions while minimizing re-clustering overhead. The runtime policy invokes these clustering mechanisms natively during initialization and continuously monitors for necessary dynamic re-assignments (Algorithm 3.1, Lines 2–5 and 19–22).

3.3 Synchronization Frequency

The third component of POWERSCALE’s policy controls how often each cluster synchronizes with the global aggregator. This is governed by the local round ratio L , the number of intra-cluster rounds between consecutive global-tier aggregations. A larger L means more local rounds between global synchronizations, which reduces the frequency of WAN transfers: over R total training rounds, the system performs only R/L global exchanges, so increasing L directly reduces total communication energy. However, fewer global synchronizations allow cluster models to diverge from the global average, potentially requiring additional rounds to reach target accuracy. The design challenge is choosing L large enough to meaningfully reduce communication energy without increasing total training rounds to the point where the additional compute energy offsets the savings. POWERSCALE supports two frequency policies.

Fixed frequency. The simplest policy assigns a single fixed L to all clusters and holds it constant throughout training. Every cluster synchronizes at the same interval regardless of its communication cost or the model’s convergence state. This policy cannot adapt to either spatial or temporal variation and serves as the lower-bound baseline.

Uniform frequency. The uniform policy also applies the same L to all clusters within each round, but allows L to change over time as training progresses. This captures temporal diminishing returns, reducing global synchronization as updates become incremental, but it cannot account for heterogeneous communication costs across clusters: a cluster with high WAN cost synchronizes at the same frequency as one with low WAN cost.

Adaptive frequency. Clusters vary in their distance to the global aggregator, and the marginal value of each global synchronization decreases as training progresses. POWERSCALE’s adaptive frequency policy accounts for both spatial heterogeneity across clusters and temporal diminishing returns over the course of training.

It first assigns each cluster a per-cluster baseline ratio proportional to its relative communication cost:

$$L_c = \min \left(L_{\max}, \max \left(1, \left\lfloor L_{\text{base}} \cdot \frac{\bar{T}_c^{\text{global}}}{\bar{T}_c^{\text{local}}} \right\rfloor \right) \right), \quad (6)$$

where L_{base} is a configurable baseline ratio shared across all sites that sets the scale of all per-cluster values, $\bar{T}_c^{\text{global}}$ and $\bar{T}_c^{\text{local}}$ are cluster c ’s mean inter- and intra-cluster transfer times (Eq. 1), and $L_{\max}$ caps the local rounds any cluster may run between global synchronizations, preventing indefinite model drift. Clusters with a high global-to-local cost ratio receive a larger L_c , synchronizing less often and spending less communication energy per round of training.

The adaptive policy then modulates each L_c over time based on observed training progress. As training matures, gradient magnitudes decrease and parameter updates become incremental [12]; paying the same WAN energy in late training as in early training wastes energy on communication that barely improves accuracy. To avoid this, cluster c defers global synchronization while its model is changing minimally, and triggers one when its accumulated local update grows large or its accuracy improvement stalls:

$$\|\theta_c^{(\ell)} - \theta_c^{(0)}\| > \delta \quad \text{or} \quad \Delta\alpha_c < \epsilon \text{ for } P_{\text{patience}} \text{ consecutive rounds,} \quad (7)$$

where $\theta_c^{(\ell)}$ is cluster c ’s model after ℓ local rounds since its last global synchronization, and $\theta_c^{(0)}$ is the model it received from the global aggregator at the start of that interval. The first condition triggers a global synchronization when the cluster’s local model has diverged significantly from the last globally aggregated model, ensuring that clusters undergoing substantial learning propagate their updates to the rest of the system before divergence becomes harmful. The second condition triggers a global synchronization when the cluster’s local accuracy has not improved by more than ϵ for P_{patience} consecutive rounds; in this case, the cluster has stalled locally and replaces its model with the current global average, which incorporates learning from all other clusters and may help the stalled cluster escape its plateau. Together, these conditions concentrate global communication in two regimes: early in training, when models change rapidly and the magnitude condition occurs

frequently, and during local stalls, when a cluster needs external progress to continue improving. As training matures and updates become incremental, neither condition occurs often, and the interval between global rounds grows naturally, reducing communication energy. The cap $L_{\max}$ guarantees that every cluster synchronizes at least once within $L_{\max}$ local rounds regardless of these conditions, placing a hard bound on model drift.

When POWERSCALE reassigns sites between clusters (Section 3.2), the affected clusters’ $\bar{T}_c^{\text{global}}$ and $\bar{T}_c^{\text{local}}$ change. After each reassignment, POWERSCALE recomputes these values from the updated membership and recalculates L_c via Eq. 6 before resuming adaptive modulation. This ensures the frequency policy remains consistent with the current cluster structure. We evaluate both policies and their interaction with dynamic clustering in Section 5.5. This adaptive frequency mechanism directly controls the condition for breaking the local training loop and triggering a global synchronization (Algorithm 3.1, Lines 6 and 11–16).

3.4 Summary

Algorithm 3.1 composes the three preceding mechanisms into a single runtime policy. The algorithm ties together initialization (clustering and L_c assignment), the training loop (synchronous inner aggregation, adaptive outer synchronization triggers, and staleness-weighted global merging), and periodic reclustering as power conditions shift.

Note that the policies outlined above assume a two-level hierarchy ($H=2$: sites $\rightarrow$ CAs $\rightarrow$ GA), placing the aggregation boundary at the WAN. Deeper hierarchies ($H > 2$) introduce additional pre-aggregation tiers that further consolidate WAN traffic, reducing per-round communication energy, but each additional tier accumulates model drift, which can increase the total rounds required to reach target accuracy. Whether the savings outweigh this cost depends on scale: at larger site counts, traffic reduction from an additional tier is more pronounced, while per-hop drift is bounded by $L_{\max}$. We adopt $H=2$ as the default and extend it to $H=3$ in Section 5.6 to characterize this trade-off. POWERSCALE does not attempt to optimize depth. Determining the H value that minimizes total energy for an arbitrary deployment is a hard joint optimization problem across network structure, power distributions, and convergence dynamics. This is out of scope and reserved for future work.

4 Implementation

POWERSCALE is implemented in Python using PyTorch for model execution and Flower-compatible client and strategy interfaces. An experiment configuration is first resolved into an explicit topology containing leaf sites, aggregation nodes, and their parent–child relationships. The runtime materializes this topology recursively: each leaf node wraps a data partition and local training task, while each internal node presents the same fit and evaluate interface as a client to its parent. This recursive representation lets the same execution path instantiate a single-tier baseline, the default two-tier hierarchy, or a deeper hierarchy without duplicating the training logic.

Each aggregation node maintains its current model, local-round counter, and synchronization state. During a local step, it distributes

Algorithm 3.1 POWERSCALE Runtime Policy

Require: N sites with distances d_{ij} and power fractions $P_{\text{frac},k}$; thresholds L_{max} , τ_P , δ , ϵ , P_{patience} ; reclustering interval $R_{\text{recluster}}$

// Initialization (Section 3.2)

- 1: Rank sites by accuracy gain per unit energy (Eq. 4); select active set
- 2: Choose K minimizing estimated per-round time; form clusters $\{C_c\}$ using joint feature f_k (Eq. 3)
- 3: Designate site nearest each cluster centroid as cluster aggregator (CA); designate GA
- 4: Assign each cluster baseline L_c (Eq. 6)

// Training loop

- 5: **for** each global round t **do**
- 6: **for** each cluster c **in parallel do**
- 7: $\ell_c \leftarrow 0$
- 8: **repeat**
- 9: CA broadcasts θ_c ; sites compute local updates; CA averages ▷ sync inner
- 10: $\ell_c \leftarrow \ell_c + 1$
- 11: **until** $\|\theta_c^{(\ell_c)} - \theta_c^{(0)}\| > \delta$ **or** $\Delta\alpha_c < \epsilon$ for P_{patience} rounds
- 12: **or** $\ell_c \geq L_{\text{max}}$
- 13: CA pushes θ_c to GA with timestamp t_c ▷ async outer
- 14: **end for**
- 15: GA incorporates each arriving θ_c into global model θ , weighted by $1/(t - t_c + 1)$
- 16: // Reclustering check
- 17: **if** $t \bmod R_{\text{recluster}} = 0$ **or** $\exists k : |P_{\text{frac},k}^{(t)} - P_{\text{frac},k}^{(\text{last})}| > \tau_P$ **then**
- 18: Migrate affected sites to nearest centroid; recompute L_c for modified clusters
- 19: **end if**
- 20: **end for**

a cloned model to its children, collects their updates, and forms a sample-weighted aggregate. Leaf clients execute the actual PyTorch training loop on their assigned partitions; a bounded worker pool runs independent sibling clients concurrently while preserving the dependency order imposed by the hierarchy. At the cluster tier, this procedure implements the synchronous inner loop. At the global tier, the runtime can pass completed cluster updates to a buffered aggregator, which merges the buffer using the configured staleness weights. The same aggregation primitives are also used by the single-tier and fixed-policy configurations, ensuring that policies differ in topology and control state rather than in the underlying model-update path.

The adaptive mechanisms are implemented as a controller separate from the training primitives. At global-round boundaries, the controller maintains active-site membership, cluster assignments, and the synchronization interval for each cluster. When a clustering update changes the hierarchy, the runtime rebuilds the affected internal aggregation nodes from the updated assignment while reusing the existing leaf clients and their data partitions. It then propagates the revised synchronization intervals to the corresponding nodes before the next round begins. A structured per-round log

Dataset	Task	Model	Classes	Samples
CIFAR-10	Image Class.	ResNet-18	10	60,000
Shakespeare	Next-Char Pred.	LSTM	80	422,615
EMNIST	Image Class.	CNN	62	814,255

Table 1: Statistics for datasets used in the evaluation.

records model metrics, elapsed execution time, participating sites, cluster members, synchronization intervals, and asynchronous-aggregation metadata. This separation keeps the adaptive policy explicit and makes every hierarchy and synchronization decision available to the analysis pipeline.

5 Evaluation

In this section, we present our evaluation methodology (Section 5.1), validate the energy inefficiency of single-tier aggregation (Section 5.2), then evaluate each component of POWERSCALE’s energy-efficiency policy in the order they appear in the design: synchronization modality (Section 5.3), clustering policy (Section 5.4), and synchronization frequency (Section 5.5). We close with generalizability across datasets and the hierarchy-depth trade-off (Section 5.6).

5.1 Evaluation Methodology

We present our evaluation methodology, covering the experimental setup, training datasets and models, baseline policies, evaluation metrics, and power and communication profiling.

Experimental setup. We evaluate POWERSCALE in an emulated environment of 100 geo-distributed sites, following the methodology of prior work [29]. Each site is assigned a distinct geographic location, network profile, and power availability. Because the number of emulated sites exceeds our physical GPUs, training tasks for multiple virtual sites share a GPU and run sequentially, while we compute a simulated wall-clock time that reflects execution under perfect parallelism. The communication time for each transfer follows Eq. 1, and the computation time scales inversely with the site’s power fraction $P_{\text{frac},k}$ (Section 2.1).

Datasets and models. Table 1 summarizes the datasets. Our primary evaluation uses CIFAR-10 [20] with a ResNet-18 [16], trained to a target accuracy of 94%. We demonstrate generalizability on the Shakespeare next-character prediction corpus with a two-layer LSTM and on EMNIST image classification with a CNN (Section 5.6). The data are partitioned IID across sites, consistent with our system model in which geo-distribution is driven by power rather than data locality (Section 3).

Baseline policies. We compare POWERSCALE against three baselines. (1) *FedAvg (single-tier)* [27]: the standard single-tier aggregation in which all sites synchronize with one global aggregator every round. This represents the communication structure of existing federated learning based geo-distributed training. (2) *PowerTrip* [29]: the state-of-the-art time-to-accuracy optimizer, which dynamically selects which sites participate and how many participate but retains single-tier aggregation. (3) *Static Hierarchy (Optimal)*: a hierarchy whose clustering, synchronization frequency, and depth are fixed to the best configuration found by exhaustive offline search under the *initial* power and network conditions.

Power and communication profiling. We model power availability by assigning each site a static power fraction drawn uniformly from $[0.1, 1.0]$. This value represents the available fraction of a site’s maximum potential computational throughput. Following prior work [29], we assume an inverse linear relationship between execution time and available power for compute-bound machine learning workloads; a site’s effective training time is its ideal time divided by its power fraction. For network profiles, each site is assigned a geographical distance drawn uniformly from 100 to 10,000 miles, with bandwidth scaling inversely proportional to distance. The model update size is configured to 1.5 GB. While real-world inter-datacenter bandwidths and compute capacities may be larger in absolute magnitude, these emulated parameters preserve the fundamental ratio between computation and communication costs. Consistent with state-dependent accelerator power models [18], we use normalized communication-to-compute and idle-to-compute power ratios of $\alpha = 0.30$ and $\beta = 0.15$, respectively.

Evaluation metrics. Our primary metric is *energy-to-accuracy*. This is the total energy consumed by the training sites to reach the target accuracy. It includes the compute, communication, and idle energy of all participating sites (Section 2.3). We also report *time-to-accuracy*. This is the wall-clock time required to reach the target accuracy. We use this metric to verify that energy savings do not degrade system performance.

5.2 End-to-End Energy Efficiency

We first evaluate whether POWERSCALE’s hierarchy improves end-to-end training efficiency. The two figures in this section show complementary views of the same experiment. Figure 4 plots test accuracy on the y -axis against wall-clock time on the x -axis; the dashed horizontal line marks the target accuracy. This view captures whether an energy-saving policy delays convergence. Figure 5 then plots the energy required to reach that same target, normalized to FedAvg, with each bar decomposed into compute, communication, and idle energy.

The convergence curves show that POWERSCALE reaches the target accuracy in nearly the same time as PowerTrip, while both substantially outperform FedAvg. Thus, the hierarchy does not obtain its energy savings by slowing training down. The energy breakdown shows the corresponding efficiency gain: POWERSCALE consumes $3.9\times$ less total energy than FedAvg and about $2.4\times$ less than PowerTrip. Because PowerTrip is the strongest single-tier baseline in this comparison, the gap between PowerTrip and POWERSCALE emphasizes the value of adding intermediate aggregation. We discuss POWERSCALE’s time- and energy-to-accuracy against the static hierarchy policy in Section 5.4.

Where the savings come from. The stacked bars in Figure 5 separate useful training work from coordination overhead. Compute energy changes modestly across policies, since each method must perform enough local training to reach the same target. The large reductions are instead in the overhead terms of Eq. 2. POWERSCALE reduces communication energy by roughly $10\times$ relative to FedAvg by replacing per-site WAN transmissions with per-cluster pre-aggregated updates. It also reduces idle energy by roughly $15\times$ by limiting synchronous waiting to regional clusters and using asynchronous aggregation across clusters.

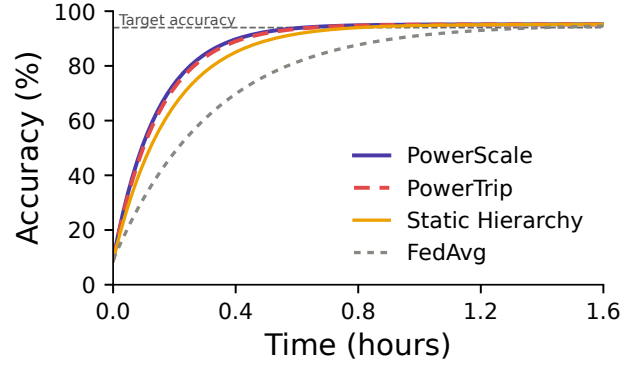


Figure 4: Test accuracy over training time for POWERSCALE and baseline policies.

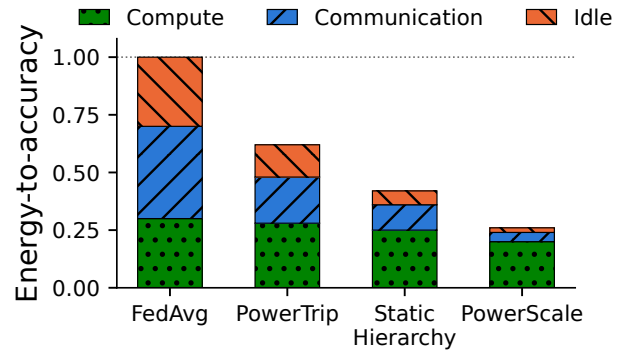


Figure 5: Energy-to-accuracy and energy component breakdown for POWERSCALE and baseline policies. Each normalized to FedAvg.

Key point. POWERSCALE preserves time-to-accuracy while reducing energy by localizing communication and idle waiting.

5.3 Synchronization Modality

Section 3.1 presented four synchronization configurations. We evaluate all four holding clustering and frequency at POWERSCALE’s defaults, to isolate the effect of the synchronization mode.

Figure 6 places the four synchronization configurations on the x -axis and reports time-to-accuracy (blue) and energy-to-accuracy (green) on the y -axis, each normalized to Sync-Sync. The dotted line at 1.0 represents the fully synchronous baseline, and lower bars are better.

Sync-Async. Changing only the outer tier from synchronous to asynchronous reduces time-to-accuracy by 31% and energy-to-accuracy by 32%. This result directly supports POWERSCALE’s Sync-Async design. A synchronous outer tier forces every cluster to wait for the slowest cluster before global progress can continue; asynchronous aggregation removes that WAN-scale barrier. Retaining synchronous aggregation inside each cluster preserves a coherent

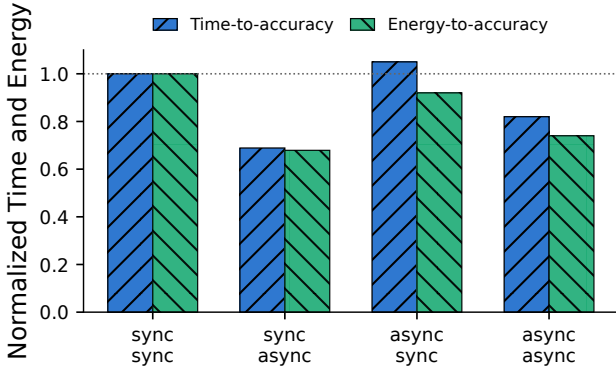


Figure 6: Energy-to-accuracy and time-to-accuracy (normalized to sync-sync) by synchronization mode.

cluster model where the participants have already been grouped to have similar completion times (Section 3.2).

Async-Sync. Removing the cluster-tier barrier allows faster sites to advance without waiting for slower members. But it also introduces intra-cluster drift that requires 5% more time to reach the target accuracy. The per-round energy saving is small because intra-cluster variance is already low, so the additional rounds yield only 8% energy savings over Sync-Sync.

Async-Async. Removing barriers at both tiers reduces time and energy relative to Sync-Sync, but drift accumulates at both levels. Compared to Sync-Async, convergence requires 14% more time and energy increases by 6%.

Key point. Removing the global tier waiting barrier cuts both time and energy by roughly one third while preserving synchronous intra-cluster training.

5.4 Clustering Policy

Clustering determines both communication and idle energy: it controls which links carry synchronization traffic and how much completion-time variance each synchronous group contains. We evaluate three decisions from Section 3.2: which clustering algorithm to use, which signals to cluster on, and whether the grouping should adapt during training.

Clustering algorithms. As argued in Section 3.2, K-means is most appropriate when the cluster count and geometric structure are largely known in advance, whereas agglomerative clustering is the better fit for a hierarchy that must be re-clustered online as power and network conditions change. To evaluate these policies, Figure 7a compares the two against a no-clustering baseline (PowerTrip’s single-tier aggregation) placed on the x-axis. The y-axis reports energy required to reach the target accuracy, normalized to the no-clustering baseline.

K-means substantially reduces energy (by 60.3%) relative to single-tier aggregation by confining most traffic to shorter intra-cluster paths and limiting straggler waiting to a smaller group of sites. Agglomerative clustering further reduces the energy by 70.2% compared to no clustering because its merge tree supports incremental repair of cluster assignments without re-solving from

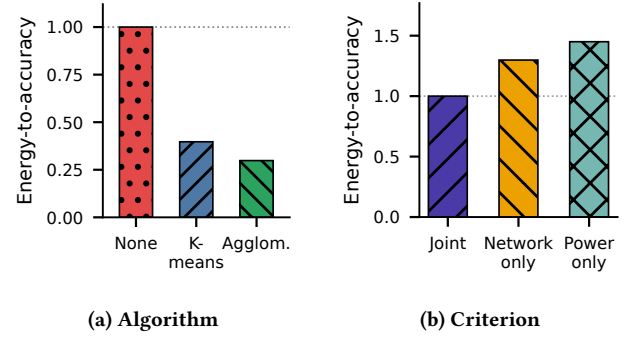


Figure 7: Energy-to-accuracy for clustering algorithms (normalized to the no-clustering baseline) (a) and clustering criteria (normalized to Joint) (b).

scratch. Thus, agglomerative reduces energy consumption by 33.2% compared to K-means. POWERSCALE therefore uses agglomerative clustering as its default.

The purpose of the experiment is not to claim that K-means is a competitive runtime policy for POWERSCALE; rather, it is to show that once a hierarchy must adapt online, agglomerative clustering is the better mechanism.

Clustering criterion. Next, we isolate the signals used to form clusters. The design in Section 3.2 combines network proximity, which controls communication cost, with available power, which controls site completion time. We compare this joint criterion against network-only and power-only baselines.

Figure 7b places the three criteria on the x-axis and reports energy to reach the target accuracy on the y-axis, normalized to joint clustering. Network-only clustering consumes 29.8% more energy and power-only clustering consumes 51.2% more energy compared with joint clustering. Network proximity alone keeps transfers local, but it cannot prevent a low-power site from pacing the synchronous inner loop. Incorporating power into the feature space forms clusters whose members are better matched in both transfer and compute time, reducing the waiting accumulated by faster sites. Conversely, the power-only result reflects the complementary risk identified in the design: balancing compute without preserving locality can place distant sites in the same cluster and increase communication cost. POWERSCALE therefore uses the joint criterion as its default.

Dynamic vs. static clustering. We compare full POWERSCALE (dynamic re-clustering) against a static baseline where the initial clustering is held fixed throughout training.

Figure 4 and Figure 5 show that dynamic re-clustering reduces energy-to-accuracy by 38.1% and time-to-accuracy by 22.5% over a static hierarchy. A fixed grouping cannot respond to power shifts, so sites whose available power drops continue to stall their clusters and accumulate idle overhead.

Key point. Clustering is substantially better than no clustering, regardless of algorithm. Joint power-and-network clustering outperforms either criterion alone. Dynamic re-clustering is essential under realistic, time-varying power.

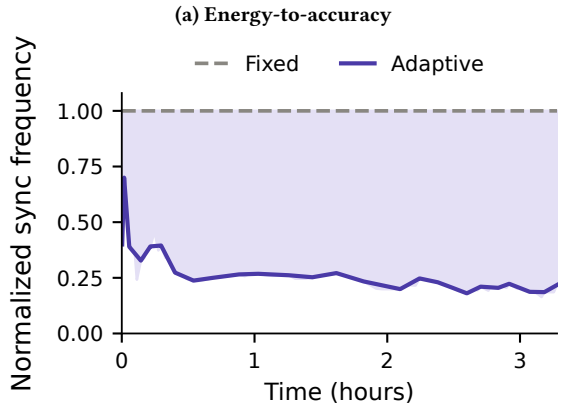
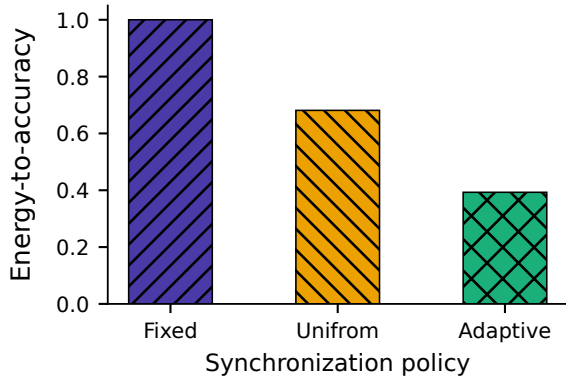


Figure 8: Energy-to-accuracy under three frequency policies (a) and global synchronization frequency over training progress (b). Both normalized to the fixed frequency policy.

5.5 Synchronization Frequency

The final policy ablation evaluates how often clusters exchange updates with the global tier. Following Section 3.3, we compare three frequency policies. The fixed policy applies the same synchronization frequency to all clusters throughout training. The uniform policy applies the same frequency to all clusters within each round but allows that frequency to change over time. The adaptive policy assigns a per-cluster frequency proportional to each cluster’s global-to-local communication cost ratio and modulates it over time based on observed convergence progress.

Figure 8a places these policies on the x -axis and reports energy-to-accuracy on the y -axis, normalized to the fixed policy. The adaptive policy reduces the energy required to achieve a given accuracy by 42.3% compared with the uniform policy and by 60.7% relative to the fixed baseline. The adaptive policy performs best because it accounts for both spatial heterogeneity across clusters and temporal diminishing returns over training.

Figure 8b illustrates the temporal distribution of energy savings throughout the training process. The x -axis is training time, while the y -axis reports effective synchronization frequency. Higher values therefore indicate more frequent coordination. The dashed gray

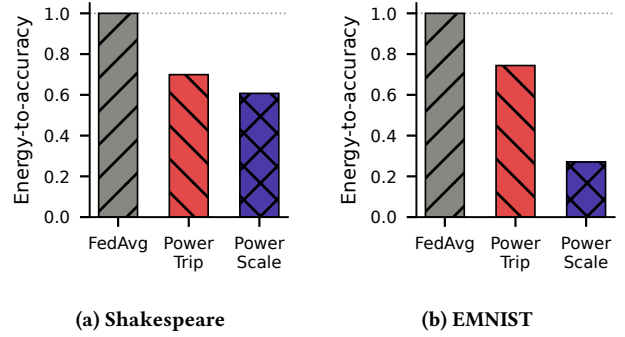


Figure 9: Energy-to-accuracy on Shakespeare (a) and EMNIST (b), normalized to FedAvg.

line is the fixed policy, while the purple line is POWERSCALE’s adaptive policy. The adaptive policy synchronizes $2.43\times$ more frequently in early training than in late training, while the fixed baseline stays flat. The shaded region between the two curves is the coordination the fixed policy continues to perform but POWERSCALE avoids: it widens as training matures, because later updates are incremental and further global synchronization adds little accuracy.

Key point. POWERSCALE’s adaptive frequency policy reduces energy by 60.7% over a fixed baseline by synchronizing less as the model converges and assigning each cluster a frequency proportional to its communication cost.

5.6 Generalizability and Hierarchy Depth

Cross-dataset generalizability. We repeat the end-to-end comparison on Shakespeare (LSTM) and EMNIST (CNN) to determine whether POWERSCALE’s savings extend beyond CIFAR-10 and ResNet-18. Figure 9a and Figure 9b report energy-to-accuracy for the three policies, normalized to FedAvg. The targets are 46% next-character accuracy for Shakespeare and 75% classification accuracy for EMNIST.

On Shakespeare, POWERSCALE uses $1.65\times$ less energy than FedAvg and $1.15\times$ less than PowerTrip. On EMNIST, the reductions are $3.70\times$ over FedAvg and $2.75\times$ over PowerTrip. The savings are larger on EMNIST because its larger model updates make communication a greater share of the single-tier baseline’s total energy, which is exactly the overhead the hierarchy removes. The policy ordering is unchanged across both workloads, confirming that the benefit extends to sequence prediction and image classification.

Hierarchy depth. The design adopts $H=2$ as the default and treats depth optimization as out of scope (Section 3.4). Nevertheless, we compare $H=2$ and $H=3$ on the 100-site scale to quantify the trade-off introduced by another aggregation tier. Figure 10 reports communication time, communication energy, energy-to-accuracy, and time-to-accuracy, each normalized to $H=2$.

The third tier reduces communication time by 47.4% and communication energy by 49.8%, lowering total energy-to-accuracy by 8.6%. These savings come with slower convergence: $H=3$ reaches the target in 35 rounds rather than 32, increasing time-to-target by 10.4%. This is the depth trade-off identified in Section 3.4: another

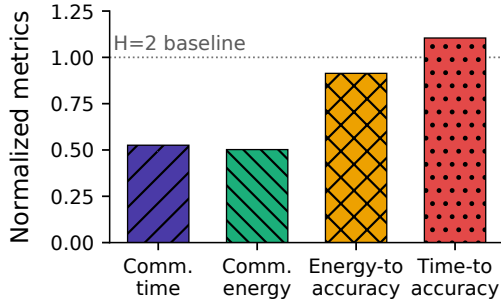


Figure 10: Effect of hierarchy depth at 100 sites. Each metric is normalized to $H=2$.

tier consolidates long-haul traffic but adds an aggregation boundary and accumulates additional model drift. At 100 sites the 8.6% energy reduction does not justify the 10.4% time penalty and the added system complexity, so $H=2$ remains the default. The communication savings grow with the number of cluster aggregators transmitting over the WAN, so deeper hierarchies become more attractive at larger scales. Characterizing that crossover requires depth runs across a range of site counts and remains future work.

6 Related Work

Communication-Efficient Distributed Training. The challenge of communication overhead in distributed training is well-established [10, 19]. Prior work focuses on reducing communication volume through gradient quantization [3], sparsification [2], and parameter-efficient fine-tuning methods like LoRA [17] and QLoRA [11]. Systems like DiLoCo [12] and Photon [37] leverage these techniques for distributed LLM training. These approaches optimize *what* is sent; POWERSCALE optimizes *how* it is sent by restructuring the communication topology.

Hierarchical Federated Learning. HFL introduces intermediate aggregation layers between edge devices and a central server [9, 24]. Prior work addresses mobile and edge constraints such as heterogeneous devices [21], privacy [14], and client-edge-cloud communication [25]. POWERSCALE adopts the structural intuition of intermediate aggregation but addresses a different trade-off: it clusters data center sites by both network cost and available power, then minimizes energy-to-accuracy under WAN-bound communication rather than optimizing an edge deployment’s battery use, privacy, or server load.

Adaptive Synchronization. Techniques such as local SGD [41] and adaptive aggregation frequency [1, 7] reduce communication by performing multiple local updates between synchronizations. Buffered asynchronous aggregation similarly mitigates stragglers by aggregating updates asynchronously in batches [31]. POWERSCALE extends these ideas to a multi-tiered setting: clusters synchronize frequently within regions, while individual clusters adapt their costly inter-cluster synchronization rate to network cost and convergence progress. Its Sync-Async option applies asynchronous aggregation only at the outer tier, retaining synchronous regional aggregation where it limits local divergence.

Resource-Constrained AI Training. Prior work addresses power and resource constraints in AI training [33, 44]. SkyPilot [45] provisions GPU resources across clouds, optimizing for cost. Cross-region training [42] analyzes distributed training across cloud regions. PowerTrip [29] optimizes the power-communication trade-off in flat topologies. POWERSCALE extends this line of work by introducing hierarchy to simultaneously optimize for time-to-accuracy and energy efficiency.

Energy-Efficient ML. EcoLearn [28] minimizes the carbon footprint of federated learning through carbon-aware client selection, and Green AI [38] advocates for energy-efficient training methods broadly. Zeus tunes job- and GPU-level configurations to navigate the energy-performance trade-off for individual DNN training jobs [46]. NetStorm [22] uses hierarchical communication to reduce WAN contention in distributed training. In contrast, POWERSCALE makes the communication hierarchy itself an energy optimization decision: it dynamically clusters power-heterogeneous sites and adapts inter-cluster synchronization to minimize total energy-to-accuracy in geo-distributed data center training.

7 Conclusion

This paper identifies and addresses the energy inefficiency of single-tier aggregation in geo-distributed, power-constrained ML training. We introduce POWERSCALE, a system that utilizes hierarchical aggregation to reduce energy through three policies: a Sync-Async modality that confines synchronous waiting to low-latency regional clusters, agglomerative clustering on a joint network-proximity and power-availability criterion, and an adaptive synchronization frequency policy that tapers global synchronization as training matures. POWERSCALE reduces energy by $3.9\times$ relative to the single-tier aggregation baseline while matching or slightly improving time-to-accuracy. Future work will investigate non-IID data distributions, optimal hierarchy depth selection, and composition with gradient compression techniques.

References

- [1] Saurabh Agarwal, Hongyi Wang, Kangwook Lee, Shivaram Venkataraman, and Dimitris Papailiopoulos. Adaptive Gradient Communication via Critical Learning Regime Identification. *MLSys*, 2021.
- [2] Alham Fikri Aji and Kenneth Heafield. Sparse Communication for Distributed Gradient Descent. *EMNLP*, 2017.
- [3] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. *NeurIPS*, 2017.
- [4] DG LiMu Andersen and IW Park. Scaling Distributed Machine Learning with the Parameter Server. *OSDI*, 2014.
- [5] Victor Avelar, Patrick Donovan, Paul Lin, Wendy Torell, and Maria A Torres Arango. The AI Disruption: Challenges and Guidance for Data Center Design, 2023.
- [6] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmao, and Nicholas D. Lane. Flower: A Friendly Federated Learning Research Framework. *arXiv*, 2020.
- [7] Chen Chen, Hong Xu, Wei Wang, Baochun Li, Bo Li, Li Chen, and Gong Zhang. Synchronize Only the Immature Parameters: Communication-Efficient Federated Learning by Freezing Parameters Adaptively. *IEEE Transactions on Parallel and Distributed Systems*, 2024.
- [8] Jianmin Chen, Xinghao Pan, Rajat Monga, Samy Bengio, and Rafal Jozefowicz. Revisiting Distributed Synchronous SGD. *arXiv*, 2016.
- [9] Lakshan Cooray, Janaka Sendanayake, Pramuditha Vithanaarachchi, and Y. H. P. P. Priyadarshana. Deep Federated Learning: A Systematic Review of Methods, Applications, and Challenges. *Frontiers in Computer Science*, 2025.
- [10] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’Aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, Quoc V. Le, and Andrew Y. Ng. Large Scale Distributed Deep Networks. *NeurIPS*, 2012.

- [11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS*, 2023.
- [12] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc'Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. DiLoCo: Distributed Low-Communication Training of Language Models. *arXiv*, 2023.
- [13] Arthur Douillard, Keith Rush, Yani Donchev, Zachary Charles, Nova Fallen, Ayush Dubey, Ionel Gog, Josef Dean, Blake Woodworth, Zachary Garrett, et al. Decoupled DiLoCo for Resilient Distributed Pre-Training, 2026.
- [14] Robin C. Geyer, Tassilo Klein, and Moin Nabi. Differentially Private Federated Learning: A Client-Level Perspective. *arXiv*, 2017.
- [15] Scott Guthrie. Infinite Scale: The Architecture behind the Azure AI Superfactory, 2025.
- [16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.
- [17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
- [18] Vijay Kandiah, Scott Peever, Mahmoud Khairy, Junrui Pan, Amogh Manjunath, Timothy G Rogers, Tor M Aamodt, and Nikos Hardavellas. AccelWattch: A Power Modeling Framework for Modern GPUs. *MICRO*, 2021.
- [19] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. Federated Learning: Strategies for Improving Communication Efficiency. *arXiv*, 2016.
- [20] Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images. *University of Toronto*, 2009.
- [21] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated Optimization in Heterogeneous Networks. *MLSys*, 2020.
- [22] Zonghang Li, Wenjiao Feng, Weibo Cai, Hongfang Yu, Long Luo, Gang Sun, Hongyang Du, and Dusit Niyato. Accelerating Geo-Distributed Machine Learning with Network-Aware Adaptive Tree and Auxiliary Route. *IEEE/ACM Transactions on Networking*, 2024.
- [23] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J. Dally. Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training. *arXiv*, 2017.
- [24] Lumin Liu, Jun Zhang, S. H. Song, and Khaled B. Letaief. Client-Edge-Cloud Hierarchical Federated Learning. *ICC*, 2020.
- [25] Long Luo, Chi Zhang, Hongfang Yu, Gang Sun, Shouxi Luo, and Schahram Dustdar. Communication-Efficient Federated Learning with Adaptive Aggregation for Heterogeneous Client-Edge-Cloud Network. *IEEE Transactions on Services Computing*, 2024.
- [26] J. B. MacQueen. Some Methods for Classification and Analysis of Multivariate Observations. *Berkeley Symposium on Mathematical Statistics and Probability*, 1967.
- [27] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. *AISTATS*, 2017.
- [28] Talha Mehboob, Noman Bashir, Jesus A. Omaña Iglesias, Michael Zink, and David Irwin. EcoLearn: Optimizing the Carbon Footprint of Federated Learning. *SEC*, 2025.
- [29] Talha Mehboob, Luanzheng Guo, Nathan Tallent, Michael Zink, and David Irwin. PowerTrip: Exploiting Federated Heterogeneous Datacenter Power for Distributed ML Training. *SoCC*, 2025.
- [30] Fionn Murtagh and Pedro Contreras. Algorithms for Hierarchical Clustering: An Overview. *WIREs Data Mining and Knowledge Discovery*, 2012.
- [31] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Michael Rabbat, Mani Malek, and Dzmitry Huba. Federated Learning with Buffered Asynchronous Aggregation. *AISTATS*, 2022.
- [32] Tyler Norris, Timothy Profeta, Dalia Patino-Echeverri, and Adam Cowie-Haskell. Rethinking Load Growth: Assessing the Potential for Integration of Large Flexible Loads in US Power Systems, 2025.
- [33] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, David Rothchild, David So, Maud Texier, and Jeff Dean. Carbon Emissions and Large Neural Network Training. *arXiv*, 2021.
- [34] Joseph Rand, Nick Manderlink, Will Gorman, Ryan H. Wiser, Joachim Seel, Julie Mulvaney Kemp, Seongeun Jeong, and Fritz Kahrl. Queued Up: 2024 Edition, Characteristics of Power Plants Seeking Transmission Interconnection as of the End of 2023, 2024.
- [35] Goldman Sachs. Generational Growth: AI, Data Centers and the Coming US Power Demand Surge, 2024.
- [36] Varun Sakalkar, Vasileios Kontorinis, David Landhuis, Shaohong Li, Darren De Ronde, Thomas Blooming, Anand Ramesh, James Kennedy, Christopher Malone, Jimmy Clidas, and Parthasarathy Ranganathan. Data Center Power Oversubscription with a Medium Voltage Power Plane and Priority-Aware Caping. *ASPLOS*, 2020.
- [37] Lorenzo Sani, Alex Jacob, Zeyu Cao, Royson Lee, Bill Marino, Yan Gao, Wanru Zhao, Dongqi Cai, Zexi Li, Xinchu Qiu, and Nicholas D. Lane. Photon: Federated LLM Pre-Training. *MLSys*, 2025.
- [38] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. Green AI. *Communications of the ACM*, 2020.
- [39] Alexander Sergeev and Mike Del Balso. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv*, 2018.
- [40] Arman Shehabi, Alex Newkirk, Sarah J. Smith, Alex Hubbard, Nuoai Lei, Md Abu Bakar Siddik, Billie Holec, Jonathan Koomey, Eric Masanet, and Dale Sartor. 2024 United States Data Center Energy Usage Report. *Lawrence Berkeley National Laboratory*, 2024.
- [41] Sebastian U. Stich. Local SGD Converges Fast and Communicates Little. *ICLR*, 2019.
- [42] Foteini Strati, Paul Elvinger, Tolga Kerimoglu, and Ana Klimovic. ML Training with Cloud GPU Shortages: Is Cross-Region the Answer? *EuroMLSys*, 2024.
- [43] U.S. Energy Information Administration. Electricity Data Browser, 2025.
- [44] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga Behram, James Huang, Charles Bai, Michael Gschwind, Anurag Gupta, Myle Ott, Anastasia Melnikov, Salvatore Candido, David Brooks, Geeta Chauhan, Benjamin Lee, Hsien-Hsin S. Lee, Bugra Akylidiz, Maximilian Balandat, Joe Spisak, Ravi Jain, Mike Rabbat, and Kim Hazelwood. Sustainable AI: Environmental Implications, Challenges and Opportunities. *MLSys*, 2022.
- [45] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, and Ion Stoica. SkyPilot: An Intercloud Broker for Sky Computing. *NSDI*, 2023.
- [46] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training. *NSDI*, 2023.